

Reviewer Pack — Real Rank of SOS Moment Matrices for Max-CUT

Entry 938d297912a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 21:28:10 UTC

Real Rank of SOS Moment Matrices for Max-CUT

Entry ID: 938d297912a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-30 21:28:10 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Sum-of-Squares Hierarchy

Statement:

For any Max-CUT instance with n variables, the real rank of its degree- d SOS moment matrix is $\Omega(n)$ if the integrality gap of the d -round SOS relaxation is > 0.878 .

Rationale (proposer’s reasoning):

The real rank of the moment matrix encodes the algebraic complexity of the feasible region. A linear lower bound on rank suggests the SOS hierarchy requires $\Omega(n)$ rounds to capture the true maximum, aligning with conjectured hardness of approximating Max-CUT beyond Goemans-Williamson.

Taxonomy category: SOS_DEGREE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 73258fc30de59e94

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        # Generate a random Max-CUT instance with n variables
        A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        for i in range(n):
            A[i][i] = 0
        return A

    def dpll(g, assignment):
        # Simple DPLL algorithm to check if a Max-CUT instance is integrality gap > 0.878
        if not g:
            return True
        v = next(v for v in range(len(g)) if v not in assignment)
        for val in [0, 1]:
            new_assignment = assignment.copy()
            new_assignment[v] = val
            if dpll([g[i][j] for j in range(len(g)) if i != j and (i < j or (i > j and new_assignment[j] != val)]), new_assignment):
                return True
        return False

    def matrix_multiply(A, B):
        # Matrix multiplication using list of lists
        m, n = len(A), len(B[0])
        p = len(B)
        C = [[sum(A[i][k] * B[k][j] for k in range(p)) for j in range(n)] for i in range(m)]
        return C

    def eigenvalue_decomposition(M):
        # Simple power iteration method to find the largest eigenvalue
        n = len(M)
        v = [1.0 / math.sqrt(n)] * n
        for _ in range(100): # Number of iterations
            v = matrix_multiply(M, v)
            norm = sum(x**2 for x in v) ** 0.5
            v = [x / norm for x in v]
        return v

    def sos_moment_matrix(A):
        n = len(A)
        M = [[0] * (n + 1) for _ in range(n + 1)]
```

```

M[0][0] = 1
for i in range(n):
    M[i + 1][i + 1] = A[i][i]
    for j in range(i + 1, n):
        M[j - i][j] = M[j][j - i] = A[i][j]
return M

def real_rank(M):
    # Real rank via eigenvalue decomposition
    eigvals = eigenvalue_decomposition(M)
    return sum(1 for x in eigvals if abs(x) > 1e-6)

n = random.choice([10, 20, 30, 40])
A = generate_max_cut_instance(n)
M = sos_moment_matrix(A)
rank = real_rank(M)

gap = dpll(A, {})
conjecture_holds = gap and rank >= 0.3 * n
counterexample = "gap_check_failed" if not gap else ""

return {
    "metric_name": "real_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [random.randint(2, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = (sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='gap_check_failed' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cfd88e.py", line 95, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cfd88e.py", line 75, in run_trial
    rank = real_rank(M)
            ^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cfd88e.py", line 69, in real_rank
    eigvals = eigenvalue_decomposition(M)
                ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cfd88e.py", line 52, in eigenvalue_decomposition
    v = matrix_multiply(M, v)
          ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8cfd88e.py", line 42, in matrix_multiply
    m, n = len(A), len(B[0])
              ^^^^^^^^^
TypeError: object of type 'float' has no len()
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error before producing results, preventing validation of conjecture | next: Fix matrix_multiply function to handle float inputs properly and re-run tests

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	48951
2	preregistration	ollama_remote	qwen3:8b	0	0	24184
3	novelty	ollama_remote	qwen3:8b	0	0	20660
4	novelty	ollama_remote	qwen3:8b	0	0	14886
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17549
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11342
7	judge	ollama_remote	qwen3:8b	0	0	12787

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 150359 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/938d297912a8.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/938d297912a8.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/938d297912a8.tar.gz` (if generated)